

Knapsack

Решение

Отжиг

Шедулинг

- температура убывает экспоненциально, множитель 0.999999
- вероятность перехода обычная, $P = \exp\left(-\frac{\Delta_{\text{score}}}{T}\right)$

Состояние

- для каждого элемента храним, взяли ли его или нет
- сумма взятых элементов

Старт

Жадный: отсортируем элементы по полезности ($\frac{\text{cost}}{\text{weight}}$), пройдемся в порядке убывания полезности и будем брать элементы, если они влезают.

Переходы

- добавить невзятый элемент (семплируется по полезности с помощью дерева Фенвика)
- удалять взятый элемент (семплируется по обратной полезности)

Кост-функция

Нормализованная сумма cost взятых элементов (со знаком минус) так, чтобы $\text{avg}(\text{cost}) = 1$.

Первая итерация

Банально берем случайный невзятый элемент или удаляем случайный взятый элемент.

Instance	Score	CPU time, s	Points
ks_30_0	96 825	1.06	3
ks_50_0	139 894	1.78	0
ks_200_0	90 343	3.83	0
ks_400_0	3 952 124	9.85	0
ks_1000_0	99 073	17.09	0
ks_10000_0	1 006 812	137.64	0

Итого: 3 / 30.

Вторая итерация

Нормализуем веса так, чтобы средний cost объекта был равен 1.

Instance	Score	CPU time, s	Points
ks_30_0	99 798	1.48	5
ks_50_0	140 676	2.66	0
ks_200_0	100 236	4.31	5
ks_400_0	3 951 494	14.90	0
ks_1000_0	104 672	25.43	0

Итого: 10 / 25.

Третья итерация

Жадный старт с сортировкой по полезности, понизил множитель с 0.999999 до 0.99999.

Instance	Score	CPU time, s	Points
ks_30_0	99 798	0.15	5
ks_50_0	141 956	0.27	3
ks_200_0	100 236	0.44	5
ks_400_0	3 966 813	1.18	3
ks_1000_0	109 869	2.43	3
ks_10000_0	1 099 870	21.75	3

Итого: 22 / 30.

Четвёртая итерация

Чтобы изучать большую долю пространства решений, переделал кандидатов: теперь есть один переход – удалить элемент и жадно набить рюкзак, не используя этот элемент.

Instance	Score	CPU time, s	Points
ks_30_0	99 752	0.12	3
ks_50_0	141 956	0.19	3
ks_200_0	100 062	0.49	3
ks_400_0	3 966 813	0.94	3
ks_1000_0	109 879	2.54	3
ks_10000_0	1 099 870	25.13	3

Итого: 18 / 30.

Изученная доля всё ещё очень маленькая, отжиг почти не рассматривает состояний.

Пятая итерация

Скомбинируем идеи из третьей и четвёртой: давайте при удалении объекта записывать его в некоторый “бан-лист”, то есть запретим брать его несколько следующих итераций.

Например, давайте сделаем бан-лист размера 3 из 3 последних удалений.

Instance	Score	CPU time, s	Points
ks_30_0	99 798	0.21	5
ks_50_0	141 956	0.33	3
ks_200_0	100 236	0.49	5
ks_400_0	3 966 813	1.34	3
ks_1000_0	109 869	2.07	3
ks_10000_0	1 099 870	12.96	3

Итого: 22 / 30.

Шестая итерация

Бан-лист очевидно не помог, поэтому давайте расширим его до 10k итераций: если мы удалили объект среди последних 10k действий, давайте запретим его брать.

Instance	Score	CPU time, s	Points
ks_30_0	99 763	0.20	3
ks_50_0	141 960	0.30	3
ks_200_0	100 236	1.11	5
ks_400_0	3 967 168	2.43	5
ks_1000_0	109 869	3.72	3
ks_10000_0	1 099 870	15.79	3

Итого: 22 / 30.

Седьмая итерация

Давайте дадим чуть больше бюджета (множитель 0.999999), потому что банлист начал помогать. 5k/20k лучше не работают, оставим 10k.

Instance	Score	CPU time, s	Points
ks_30_0	99 798	2.05	5
ks_50_0	142 156	3.90	5
ks_200_0	100 236	12.81	5
ks_400_0	3 967 180	29.22	5
ks_1000_0	109 869	30.15	3
ks_10000_0	1 099 870	140.04	3

Итого: 26 / 30.

Восьмая итерация

Раз банлист нам так классно помогает, давайте вместо него изредка удалять сразу много случайных элементов (например, 10% по весу) и набирать новые.

Вероятности сделаем такими, чтобы в среднем размер рюкзака не менялся.

Instance	Score	CPU time, s	Points
ks_30_0	99 798	2.14	5
ks_50_0	141 956	7.11	3
ks_200_0	100 236	6.24	5
ks_400_0	3 966 813	44.00	3
ks_1000_0	109 869	67.26	3

Итого на пяти тестах: 19 / 25. Результат уже видно, что хуже банлиста, поэтому откатим.

Девятая итерация

Давайте попробуем набирать маленький остаток веса через предпосчитанное dp .